code_CHINTALA_YASHASVI_YYC3.Rmd

Yashasvi Chintala

2026-04-15

##Preprocessing

```
library(dplyr)
library(stringr)

#Loading data
df_train <- read.csv("df_train.csv", stringsAsFactors = FALSE)
df_test  <- read.csv("df-test-1.csv", stringsAsFactors = FALSE)

#Check for null data for all cols
#colSums(is.na(df_train)) # no null values were found.
#colSums(is.na(df_test))

# 2. Basic string cleaning-> col 1 and 2.
clean_text_field <- function(x) {
  x <- str_trim(x)
  x <- str_squish(x)
  x <- str_to_lower(x)
  x[x %in% c("", "na", "n/a", "null", "none", "missing")] <- NA
  x
}

df_train$neighbourhood_cleansed <- clean_text_field(df_train$neighbourhood_cleansed)
df_train$room_type <- clean_text_field(df_train$room_type)

df_test$neighbourhood_cleansed <- clean_text_field(df_test$neighbourhood_cleansed)
df_test$room_type <- clean_text_field(df_test$room_type)

#Enforcing 1st and 2nd cols to strict strings.
df_train$neighbourhood_cleansed <- as.character(df_train$neighbourhood_cleansed)
df_train$room_type <- as.character(df_train$room_type)

df_test$neighbourhood_cleansed <- as.character(df_test$neighbourhood_cleansed)
df_test$room_type <- as.character(df_test$room_type)


#Clean room_type
bad_room_type_values <- c(
  "pr", "priv", "private", "ent", "ent house", "shared", "hotel"
)

fix_room_type <- function(x) {
```

```r
  x[nchar(x) <= 1] <- NA
  x[x %in% bad_room_type_values] <- NA
  x
}

df_train$room_type <- fix_room_type(df_train$room_type)
df_test$room_type <- fix_room_type(df_test$room_type)

# ------------------------------------------------------------------------------------------------------
#Basic integer checking and cleaning.
integer_vars_train <- c(
  "bathrooms",
  "bedrooms",
  "accommodates",
  "minimum_nights",
  "availability_365",
  "number_of_reviews",
  "price",
  "calculated_host_listings_count"
)

integer_vars_test <- c(
  "bathrooms",
  "bedrooms",
  "accommodates",
  "minimum_nights",
  "availability_365",
  "number_of_reviews",
  "calculated_host_listings_count"
)

for (var in integer_vars_train) {
  df_train[[var]] <- as.numeric(df_train[[var]])
  df_train[[var]] <- trunc(df_train[[var]])
  df_train[[var]][df_train[[var]] < 0] <- NA
}

for (var in integer_vars_test) {
  df_test[[var]] <- as.numeric(df_test[[var]])
  df_test[[var]] <- trunc(df_test[[var]])
  df_test[[var]][df_test[[var]] < 0] <- NA
}

# These variables must be at least 1
must_be_at_least_1_train <- c(
  "bedrooms",
  "bathrooms",
  "accommodates",
  "minimum_nights",
  "price"
)

must_be_at_least_1_test <- c(
```

```r
  "bedrooms",
  "bathrooms",
  "accommodates",
  "minimum_nights"
)

for (var in must_be_at_least_1_train) {
  df_train[[var]][df_train[[var]] < 1] <- NA
}

#for (var in must_be_at_least_1_test) {
  #df_test[[var]][df_test[[var]] < 1] <- NA
#}

#Cleaning the review section.
review_vars <- c(
  "review_scores_rating",
  "review_scores_cleanliness",
  "review_scores_location"
)

for (var in review_vars) {
  df_train[[var]] <- as.numeric(df_train[[var]])
  df_train[[var]][df_train[[var]] < 1 | df_train[[var]] > 5] <- NA
}

for (var in review_vars) {
  df_test[[var]] <- as.numeric(df_test[[var]])
  df_test[[var]][df_test[[var]] < 1 | df_test[[var]] > 5] <- NA
}

#Cleaning Binary cols.
binary_vars <- c(
  "host_is_superhost",
  "instant_bookable",
  "host_identity_verified"
)

for (var in binary_vars) {
  df_train[[var]] <- as.numeric(df_train[[var]])
  df_train[[var]][!(df_train[[var]] %in% c(0, 1))] <- NA
}

for (var in binary_vars) {
  df_test[[var]] <- as.numeric(df_test[[var]])
  df_test[[var]][!(df_test[[var]] %in% c(0, 1))] <- NA
}

# minimum_nights should be at most 30
df_train$minimum_nights[df_train$minimum_nights > 30] <- NA
#df_test$minimum_nights[df_test$minimum_nights > 30] <- NA
#sum(is.na(df_train$minimum_nights))
```

```r
#Price over 2701 is automatically NA
df_train$price[df_train$price > 2701] <- NA
#-----------------------------------------------------------------------------------------------------
# Drop rows with NA in any column
df_train <- df_train %>%
  filter(complete.cases(.))

#df_test <- df_test %>%
  #filter(complete.cases(.))

# Strictly remove-> there are other ways, but given that there are only less than 50 of such enteries i
#View(df_test) #-> This shows all the code is proper.
#summary(df_train[review_vars]) #Between 1 and 5.
```

###Linear/polynomial Models

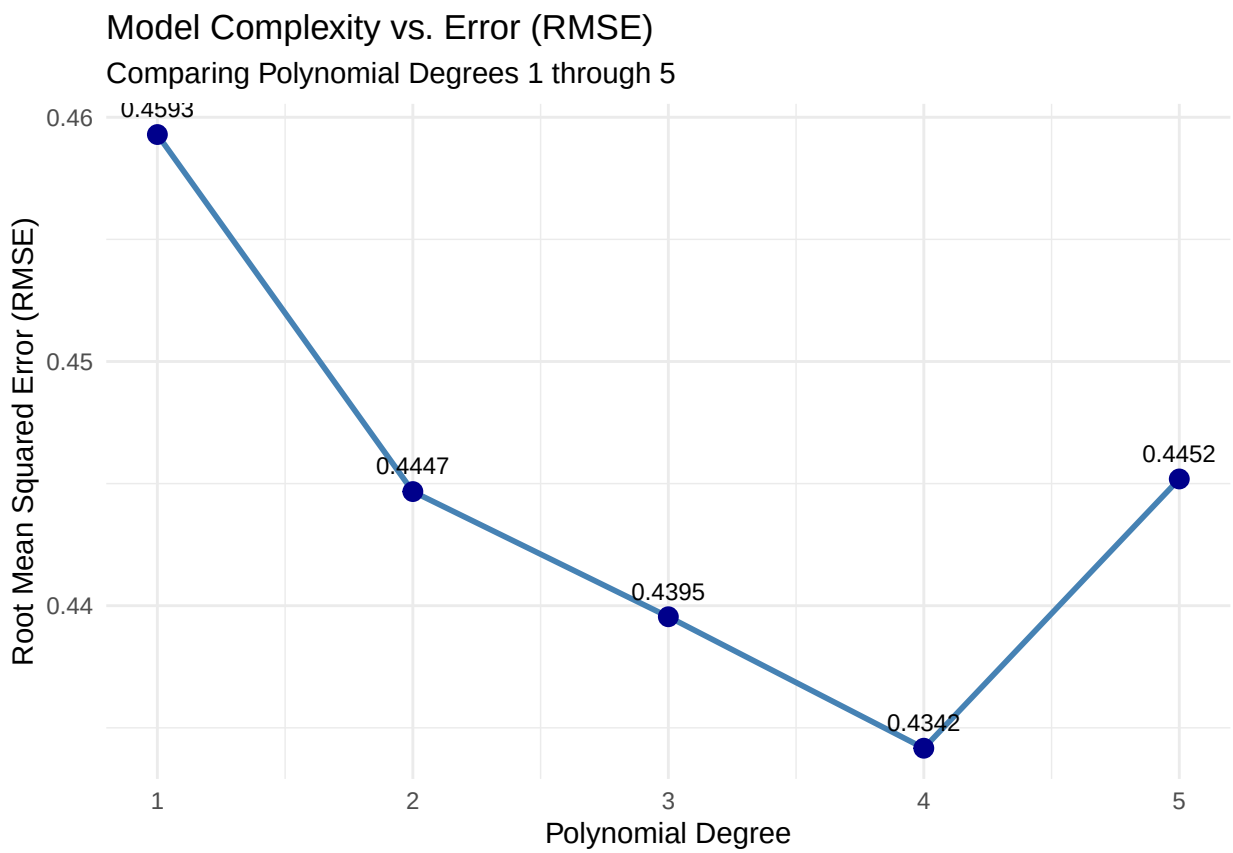
##A polynomial regression with increasing model complexities

```r
library(caret)
library(ggplot2)
train_control <- trainControl(
  method = "cv",
  number = 10
)

degrees <- 1:5

poly_results <- data.frame(
  degree = integer(),
  RMSE = numeric(),
  MSE = numeric(),
  Rsquared = numeric(),
  MAE = numeric()
)

poly_models <- list()

for (d in degrees) {

  poly_formula <- as.formula(
    paste(
      "log(price) ~ neighbourhood_cleansed + room_type +",
      "poly(bathrooms,", d, ", raw = TRUE) +",
      "poly(bedrooms,", d, ", raw = TRUE) +",
      "poly(accommodates,", d, ", raw = TRUE) +",
      "poly(minimum_nights,", d, ", raw = TRUE) +",
      "poly(availability_365,", d, ", raw = TRUE) +",
      "poly(number_of_reviews,", d, ", raw = TRUE) +",
      "poly(review_scores_rating,", d, ", raw = TRUE) +",
      "poly(review_scores_cleanliness,", d, ", raw = TRUE) +",
      "poly(review_scores_location,", d, ", raw = TRUE) +",
      "poly(calculated_host_listings_count,", d, ", raw = TRUE) +",
      "poly(dist_to_downtown,", d, ", raw = TRUE) +",
      "host_is_superhost + instant_bookable + host_identity_verified"
```

```r
    )
  )

  poly_model <- train(
    poly_formula,
    data = df_train,
    method = "lm",
    trControl = train_control,
    metric = "RMSE"
  )

  poly_models[[paste0("degree_", d)]] <- poly_model

  poly_results <- rbind(
    poly_results,
    data.frame(
      degree = d,
      RMSE = poly_model$results$RMSE,
      MSE = (poly_model$results$RMSE)^2,
      Rsquared = poly_model$results$Rsquared,
      MAE = poly_model$results$MAE
    )
  )
}
poly_results[order(poly_results$MSE), ]
```

```
##   degree      RMSE       MSE  Rsquared       MAE
## 4      4 0.4341651 0.1884993 0.6603376 0.3238375
## 3      3 0.4395478 0.1932023 0.6529545 0.3305970
## 2      2 0.4446733 0.1977343 0.6442082 0.3344830
## 5      5 0.4451892 0.1981934 0.6454839 0.3242256
## 1      1 0.4592923 0.2109494 0.6188760 0.3459925
```

```r
#Visualization
# 1. Plotting RMSE/MSE vs. Polynomial Degree
ggplot(poly_results, aes(x = degree, y = RMSE)) +
  geom_line(color = "steelblue", size = 1) +
  geom_point(size = 3, color = "darkblue") +
  geom_text(aes(label = round(RMSE, 4)), vjust = -1, size = 3) +
  labs(title = "Model Complexity vs. Error (RMSE)",
       subtitle = "Comparing Polynomial Degrees 1 through 5",
       x = "Polynomial Degree",
       y = "Root Mean Squared Error (RMSE)") +
  theme_minimal()
```



##Ridge regression

```
library(caret)
library(glmnet)

set.seed(1361)

train_control <- trainControl(
  method = "cv",
  number = 10
)

ridge_grid <- expand.grid(
  alpha = 0,
  lambda = 10^seq(-4, 4, length.out = 100)
)
ridge_cv <- train(
  log(price) ~ neighbourhood_cleansed + room_type +
    bathrooms + bedrooms + accommodates + minimum_nights +
    availability_365 + number_of_reviews + review_scores_rating +
    review_scores_cleanliness + review_scores_location +
    host_is_superhost + instant_bookable + host_identity_verified +
    calculated_host_listings_count + dist_to_downtown,
  data = df_train,
  method = "glmnet",
  trControl = train_control,
```

```r
  tuneGrid = ridge_grid,
  metric = "RMSE",
  preProcess = c("center", "scale")
)
print(min(ridge_cv$results$RMSE)^2)
```

```
## [1] 0.2108497
```

```r
#coef(ridge_cv$finalModel, ridge_cv$bestTune$lambda)

log_preds_ridge <- predict(ridge_cv, newdata = df_test)
final_predictions_ridge <- exp(log_preds_ridge)
head(final_predictions_ridge)
```

```
##         1         2         3         4         5         6
##  59.71234 135.69753 207.86279 110.22220  68.95427 322.98443
```

##Lasso regression.

```r
library(caret)
library(glmnet)

set.seed(1361)

train_control <- trainControl(
  method = "cv",
  number = 10
)

lasso_grid <- expand.grid(
  alpha = 1,
  lambda = 10^seq(-4, 4, length.out = 100)
)
#log price to make sure any missed outliers do not impact the data much, it's to make sure the model is
lasso_cv <- train(
  log(price) ~ neighbourhood_cleansed + room_type +
    bathrooms + bedrooms + accommodates + minimum_nights +
    availability_365 + number_of_reviews + review_scores_rating +
    review_scores_cleanliness + review_scores_location +
    host_is_superhost + instant_bookable + host_identity_verified +
    calculated_host_listings_count + dist_to_downtown,
  data = df_train,
  method = "glmnet",
  trControl = train_control,
  tuneGrid = lasso_grid,
  metric = "RMSE",
  preProcess = c("center", "scale")
)

print(min(lasso_cv$results$RMSE)^2)
```

```
## [1] 0.2097291
```

```r
#coef(lasso_cv$finalModel, lasso_cv$bestTune$lambda)

log_preds <- predict(lasso_cv, newdata = df_test)
final_predictions <- exp(log_preds)
head(final_predictions)
```

```
##         1         2         3         4         5         6
##  58.53109 138.13806 209.93322 108.84123  68.17937 341.06690
```

#Lasso with 2-way interaction terms.

```r
library(caret)
library(glmnet)
library(ggplot2)

set.seed(1361)

train_control <- trainControl(
  method = "cv",
  number = 10
)
lasso_grid <- expand.grid(
  alpha = 1,
  lambda = 10^seq(-5, 1, length.out = 100)
)

# Training with Degree 4 Polynomials and the full list of 18 interaction terms
lasso_cv <- train(
  log(price) ~
    neighbourhood_cleansed + room_type + host_identity_verified +
    poly(bathrooms, 4, raw = TRUE) +
    poly(bedrooms, 4, raw = TRUE) +
    poly(accommodates, 4, raw = TRUE) +
    poly(minimum_nights, 4, raw = TRUE) +
    poly(availability_365, 4, raw = TRUE) +
    poly(number_of_reviews, 4, raw = TRUE) +
    poly(review_scores_rating, 4, raw = TRUE) +
    poly(review_scores_cleanliness, 4, raw = TRUE) +
    poly(review_scores_location, 4, raw = TRUE) +
    poly(calculated_host_listings_count, 4, raw = TRUE) +
    poly(dist_to_downtown, 4, raw = TRUE) +
    host_is_superhost + instant_bookable +
    #interaction terms
    neighbourhood_cleansed:room_type +
    neighbourhood_cleansed:bathrooms +
    neighbourhood_cleansed:bedrooms +
    neighbourhood_cleansed:accommodates +
    neighbourhood_cleansed:dist_to_downtown +
    room_type:bathrooms +
    room_type:bedrooms +
    room_type:accommodates +
    room_type:dist_to_downtown +
    room_type:instant_bookable +
```

```r
    bathrooms:bedrooms +
    bathrooms:accommodates +
    bathrooms:review_scores_rating +
    bathrooms:review_scores_cleanliness +
    bedrooms:accommodates +
    bedrooms:dist_to_downtown +
    accommodates:dist_to_downtown +
    review_scores_rating:review_scores_cleanliness,
  data = df_train,
  method = "glmnet",
  trControl = train_control,
  tuneGrid = lasso_grid,
  metric = "RMSE",
  preProcess = c("center", "scale")
)
print(min(lasso_cv$results$RMSE)^2)
```

```
## [1] 0.1826547
```

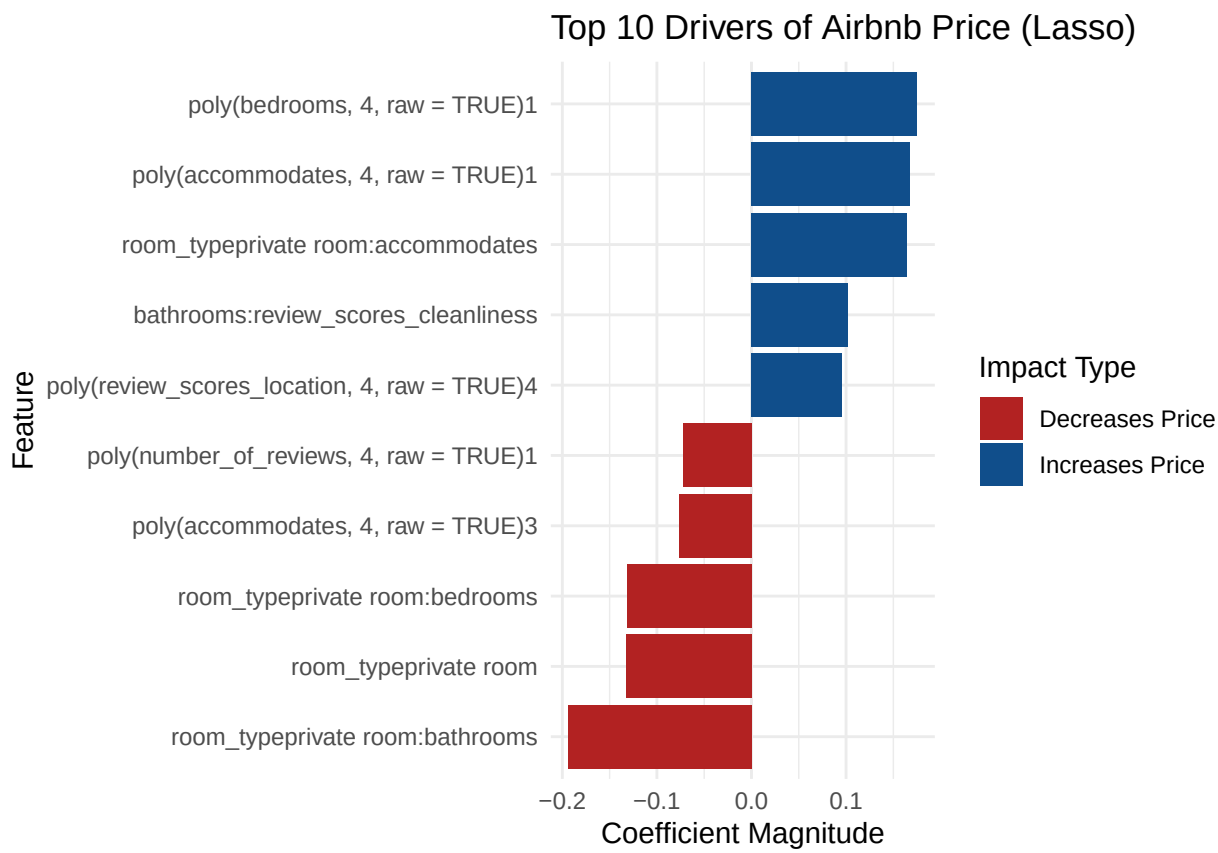
```r
#-------------- for tech report---------
# --- FOR THE TECHNICAL REPORT ---
# Best Lasso tuning results (lowest RMSE)
lasso_tech <- lasso_cv$results %>%
  arrange(RMSE) %>%
  head(10)

print(lasso_tech)
```

```
##    alpha       lambda      RMSE  Rsquared       MAE     RMSESD RsquaredSD
## 1      1 0.0017475284 0.4273812 0.6702291 0.3217304 0.04107167 0.04889740
## 2      1 0.0015199111 0.4273973 0.6702806 0.3216727 0.04096736 0.04876702
## 3      1 0.0023101297 0.4274378 0.6700831 0.3217772 0.04067573 0.04832083
## 4      1 0.0020092330 0.4274594 0.6700648 0.3217907 0.04094724 0.04870521
## 5      1 0.0013219411 0.4274852 0.6702032 0.3216964 0.04083149 0.04858455
## 6      1 0.0011497570 0.4274992 0.6702216 0.3217239 0.04067657 0.04840097
## 7      1 0.0026560878 0.4275247 0.6699426 0.3218871 0.04046137 0.04799808
## 8      1 0.0010000000 0.4276404 0.6700603 0.3218590 0.04061942 0.04838723
## 9      1 0.0006579332 0.4276990 0.6700863 0.3217437 0.04044059 0.04845170
## 10     1 0.0007564633 0.4277233 0.6700053 0.3218130 0.04054172 0.04853699
##         MAESD
## 1  0.02240007
## 2  0.02228341
## 3  0.02223740
## 4  0.02234101
## 5  0.02225632
## 6  0.02228889
## 7  0.02221010
## 8  0.02236022
## 9  0.02237560
## 10 0.02246434
```

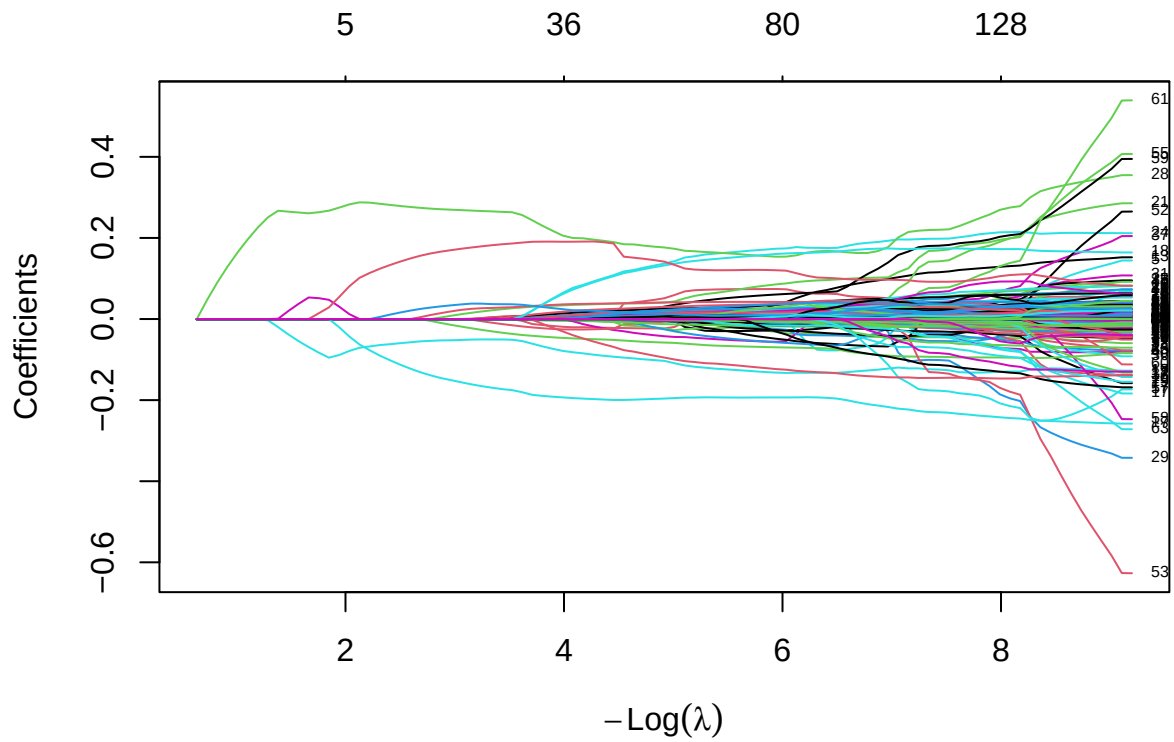
```r
#---------- for non tech report----------
# --- FOR THE NON-TECHNICAL REPORT ---
# 1. Get the coefficients for the best lambda
best_lambda <- lasso_cv$bestTune$lambda
lasso_coefs <- as.matrix(coef(lasso_cv$finalModel, s = best_lambda))

# 2. Convert to a data frame for easier sorting
lasso_df <- data.frame(
  Variable = rownames(lasso_coefs),
  Coefficient = as.numeric(lasso_coefs)
)
# 3. Filter out the zeros and sort by absolute impact
lasso_top_10 <- lasso_df %>%
  filter(Coefficient != 0 & Variable != "(Intercept)") %>%
  mutate(Abs_Impact = abs(Coefficient)) %>%
  arrange(desc(Abs_Impact)) %>%
  head(10)

print(lasso_top_10)
```

```
##                                        Variable Coefficient Abs_Impact
## 1                room_typeprivate room:bathrooms -0.19369908 0.19369908
## 2                 poly(bedrooms, 4, raw = TRUE)1  0.17543313 0.17543313
## 3             poly(accommodates, 4, raw = TRUE)1  0.16735877 0.16735877
## 4             room_typeprivate room:accommodates  0.16443293 0.16443293
## 5                         room_typeprivate room -0.13265706 0.13265706
## 6                 room_typeprivate room:bedrooms -0.13136222 0.13136222
## 7            bathrooms:review_scores_cleanliness  0.10211392 0.10211392
## 8  poly(review_scores_location, 4, raw = TRUE)4  0.09560352 0.09560352
## 9             poly(accommodates, 4, raw = TRUE)3 -0.07630046 0.07630046
## 10      poly(number_of_reviews, 4, raw = TRUE)1 -0.07199662 0.07199662
```

```r
#----------------------Visualization-----------------------

#Non tech report
# 1. Prepare the Top 10 Data (assuming you ran the filtering code from the previous step)
lasso_top_10 <- lasso_df %>%
  filter(Coefficient != 0 & Variable != "(Intercept)") %>%
  mutate(Abs_Impact = abs(Coefficient)) %>%
  arrange(desc(Abs_Impact)) %>%
  head(10)

# 2. Visualize the top drivers
ggplot(lasso_top_10, aes(x = reorder(Variable, Coefficient), y = Coefficient, fill = Coefficient > 0)) +
  geom_bar(stat = "identity") +
  coord_flip() +
  scale_fill_manual(values = c("firebrick", "dodgerblue4"), labels = c("Decreases Price", "Increases Pri
  labs(title = "Top 10 Drivers of Airbnb Price (Lasso)",
       x = "Feature",
       y = "Coefficient Magnitude",
       fill = "Impact Type") +
  theme_minimal()
```




```r
#tech report
# Plot the coefficient path from the final model
plot(lasso_cv$finalModel, xvar = "lambda", label = TRUE)
abline(v = log(lasso_cv$bestTune$lambda), col = "red", lty = 2)
```




```
#-------------- Final price coversion---------------

#coef(lasso_cv$finalModel, lasso_cv$bestTune$lambda)
log_preds <- predict(lasso_cv, newdata = df_test)
final_predictions <- exp(log_preds) # Back-transforming from log to actual dollars
#head(final_predictions)
```

###Trees and ensembles

##Random Forests

```
library(caret)
library(randomForest)

set.seed(1361)

train_control <- trainControl(
  method = "cv",
  number = 10
)

rf_grid <- expand.grid(
  mtry = c(2, 4, 6, 8, 10, 12)
)

rf_cv <- train(
```

```r
  log(price) ~ neighbourhood_cleansed + room_type +
    bathrooms + bedrooms + accommodates + minimum_nights +
    availability_365 + number_of_reviews + review_scores_rating +
    review_scores_cleanliness + review_scores_location +
    host_is_superhost + instant_bookable + host_identity_verified +
    calculated_host_listings_count + dist_to_downtown,
  data = df_train,
  method = "rf",
  trControl = train_control,
  tuneGrid = rf_grid,
  metric = "RMSE",
  importance = TRUE,
  ntree = 500
)
#rf_cv$results
rf_mse <- min(rf_cv$results$RMSE)^2
rf_mse
```

```
## [1] 0.158305
```

```r
# Top 10 RF configurations
top_rf_technical <- rf_cv$results %>%
  arrange(RMSE) %>%
  head(10)
print(top_rf_technical)
```

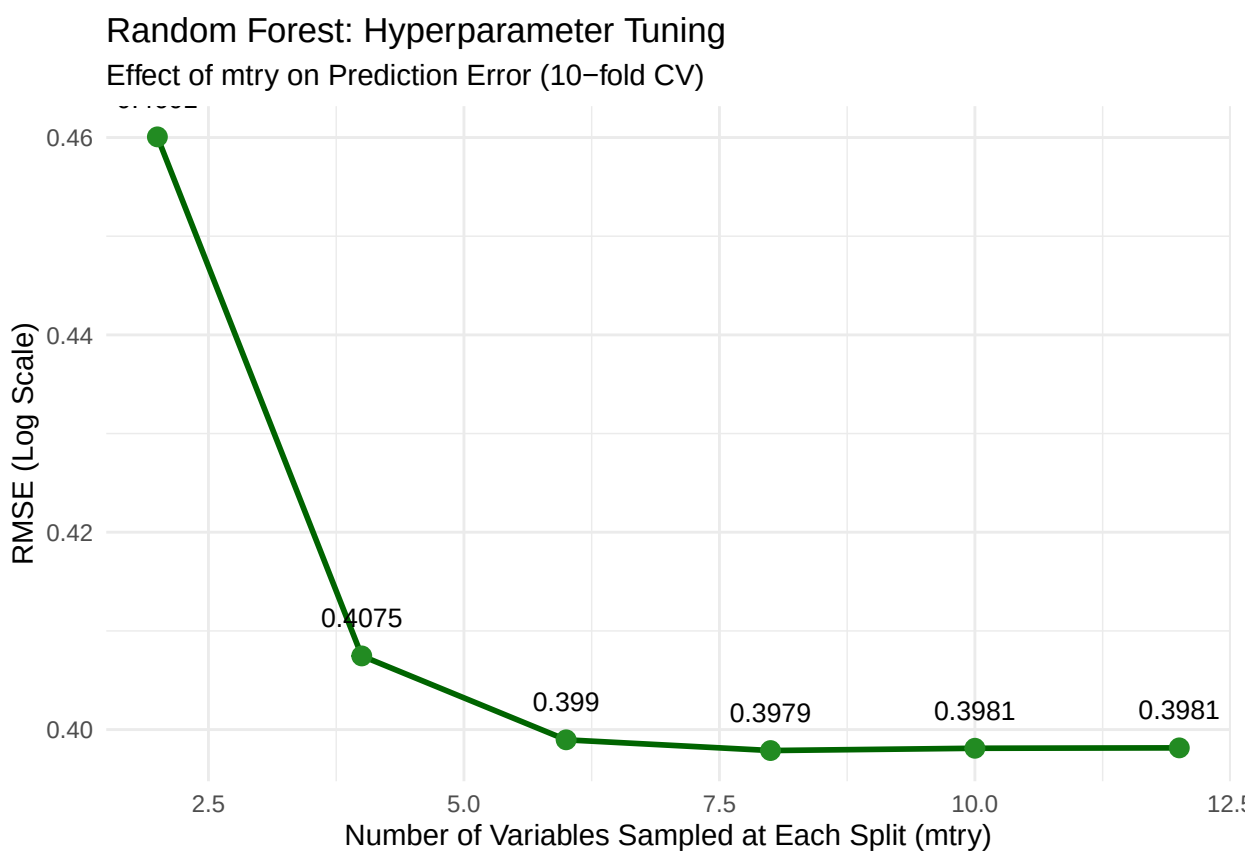
```
##   mtry      RMSE  Rsquared       MAE     RMSESD RsquaredSD      MAESD
## 1    8 0.3978756 0.7166874 0.2954028 0.04017055 0.04434948 0.02418286
## 2   10 0.3981069 0.7154362 0.2949782 0.04047643 0.04504064 0.02467377
## 3   12 0.3981463 0.7149676 0.2951245 0.03913979 0.04349445 0.02368809
## 4    6 0.3989629 0.7172543 0.2966824 0.03942717 0.04370389 0.02381800
## 5    4 0.4074634 0.7117071 0.3046923 0.03908653 0.04379303 0.02448396
## 6    2 0.4600655 0.6826677 0.3483357 0.03321849 0.04346683 0.02023330
```

```r
# Extracting RF Variable Importance
rf_importance <- varImp(rf_cv, scale = FALSE)
# Only showing top 10 for readability
print(rf_importance, 10)
```

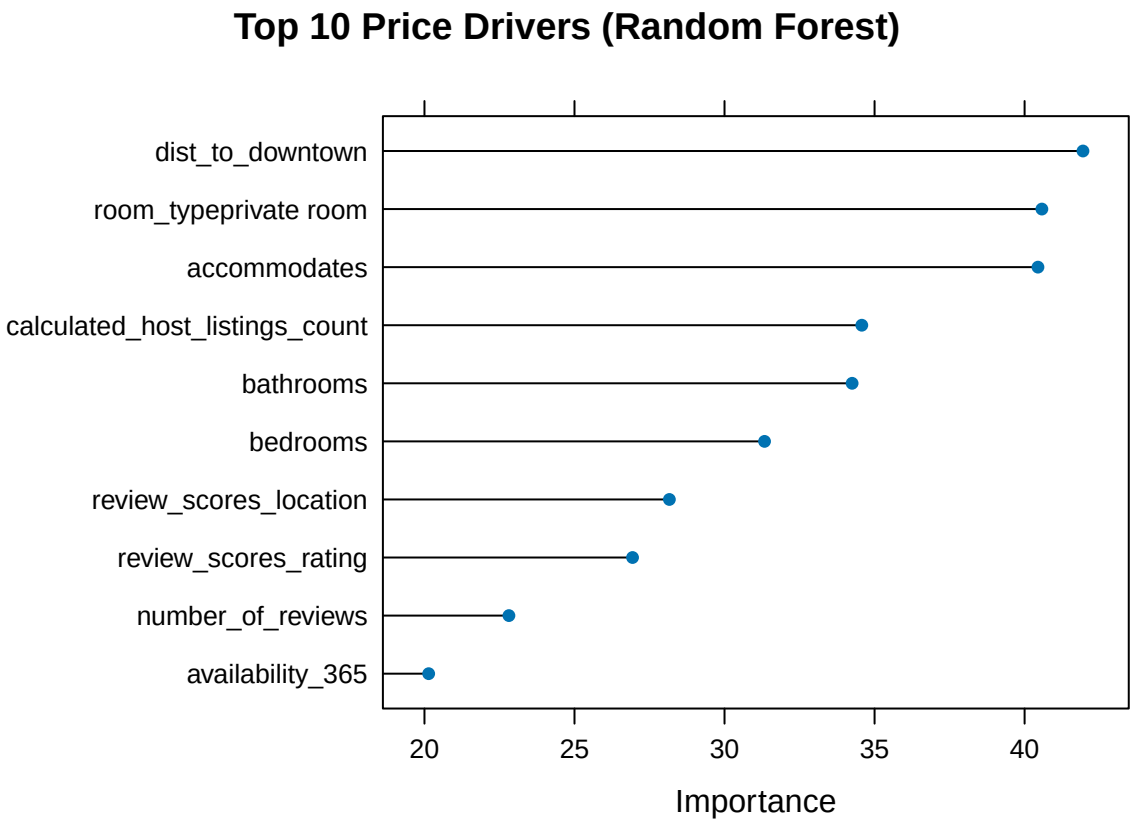
```
## rf variable importance
##
##   only 10 most important variables shown (out of 32)
##
##                                Overall
## dist_to_downtown                 41.94
## room_typeprivate room            40.58
## accommodates                     40.45
## calculated_host_listings_count   34.57
## bathrooms                        34.25
## bedrooms                         31.33
## review_scores_location           28.16
## review_scores_rating             26.93
```

```
## number_of_reviews                22.81
## availability_365                 20.14
```

```r
#----------------Visualization-----------------------
#tech report
library(ggplot2)

# Visualization of mtry vs. RMSE
ggplot(rf_cv$results, aes(x = mtry, y = RMSE)) +
  geom_line(color = "darkgreen", size = 1) +
  geom_point(color = "forestgreen", size = 3) +
  geom_text(aes(label = round(RMSE, 4)), vjust = -1.5, size = 3.5) +
  labs(title = "Random Forest: Hyperparameter Tuning",
       subtitle = "Effect of mtry on Prediction Error (10-fold CV)",
       x = "Number of Variables Sampled at Each Split (mtry)",
       y = "RMSE (Log Scale)") +
  theme_minimal()
```




```r
#non tech report
# Extract Variable Importance
rf_imp <- varImp(rf_cv, scale = FALSE)
# Visualize the Top 10 Drivers
plot(rf_imp, top = 10, main = "Top 10 Price Drivers (Random Forest)")
```




```
#-------------------------------------------------------------
log_preds_rf <- predict(rf_cv, newdata = df_test)
price_preds_rf <- exp(log_preds_rf)

original_test <- read.csv("df-test-1.csv", stringsAsFactors = FALSE)
final_ids <- original_test$ID
submission_final <- data.frame(
  id = final_ids,
  price = price_preds_rf
)
if(nrow(submission_final) == 700) {
  write.csv(submission_final, "testing_predictions_CHINTALA_YASHASVI_YYC3.csv", row.names = FALSE)
} else {
  print("!700")
}
```

##Bagging

```
library(caret)
library(randomForest)

set.seed(1361)

train_control <- trainControl(
  method = "cv",
  number = 10
```

```r
)

p <- 16

bagging_grid <- expand.grid(mtry = p)

bagging_cv <- train(
  log(price) ~ neighbourhood_cleansed + room_type +
    bathrooms + bedrooms + accommodates + minimum_nights +
    availability_365 + number_of_reviews + review_scores_rating +
    review_scores_cleanliness + review_scores_location +
    host_is_superhost + instant_bookable + host_identity_verified +
    calculated_host_listings_count + dist_to_downtown,
  data = df_train,
  method = "rf",
  trControl = train_control,
  tuneGrid = bagging_grid,
  metric = "RMSE",
  ntree = 500
)

bagging_cv$results
```

```
##   mtry     RMSE Rsquared       MAE    RMSESD RsquaredSD      MAESD
## 1   16 0.401305  0.70997 0.2972666 0.0398831 0.04492415 0.02481289
```

```r
bagging_mse <- min(bagging_cv$results$RMSE)^2
bagging_mse
```

```
## [1] 0.1610457
```

##Gradient Boosting

```r
library(caret)
library(gbm)
library(dplyr)
library(ggplot2)

set.seed(1361)

train_control <- trainControl(
  method = "cv",
  number = 10
)

gbm_grid <- expand.grid(
  n.trees = c(100, 300, 500),
  interaction.depth = c(1, 2, 3, 4),
  shrinkage = c(0.01, 0.05, 0.1),
  n.minobsinnode = c(5, 10)
)
```

```r
gbm_cv <- train(
  log(price) ~ neighbourhood_cleansed + room_type +
    bathrooms + bedrooms + accommodates + minimum_nights +
    availability_365 + number_of_reviews + review_scores_rating +
    review_scores_cleanliness + review_scores_location +
    host_is_superhost + instant_bookable + host_identity_verified +
    calculated_host_listings_count + dist_to_downtown,
  data = df_train,
  method = "gbm",
  trControl = train_control,
  tuneGrid = gbm_grid,
  metric = "RMSE",
  verbose = FALSE
)

#gbm_cv$results
best_gbm_mse <- min(gbm_cv$results$RMSE)^2
#best_gbm_mse

# Top 10 models with the lowest RMSE
top_gbm <- gbm_cv$results %>%
  arrange(RMSE) %>%
  head(10)
print(top_gbm)
```

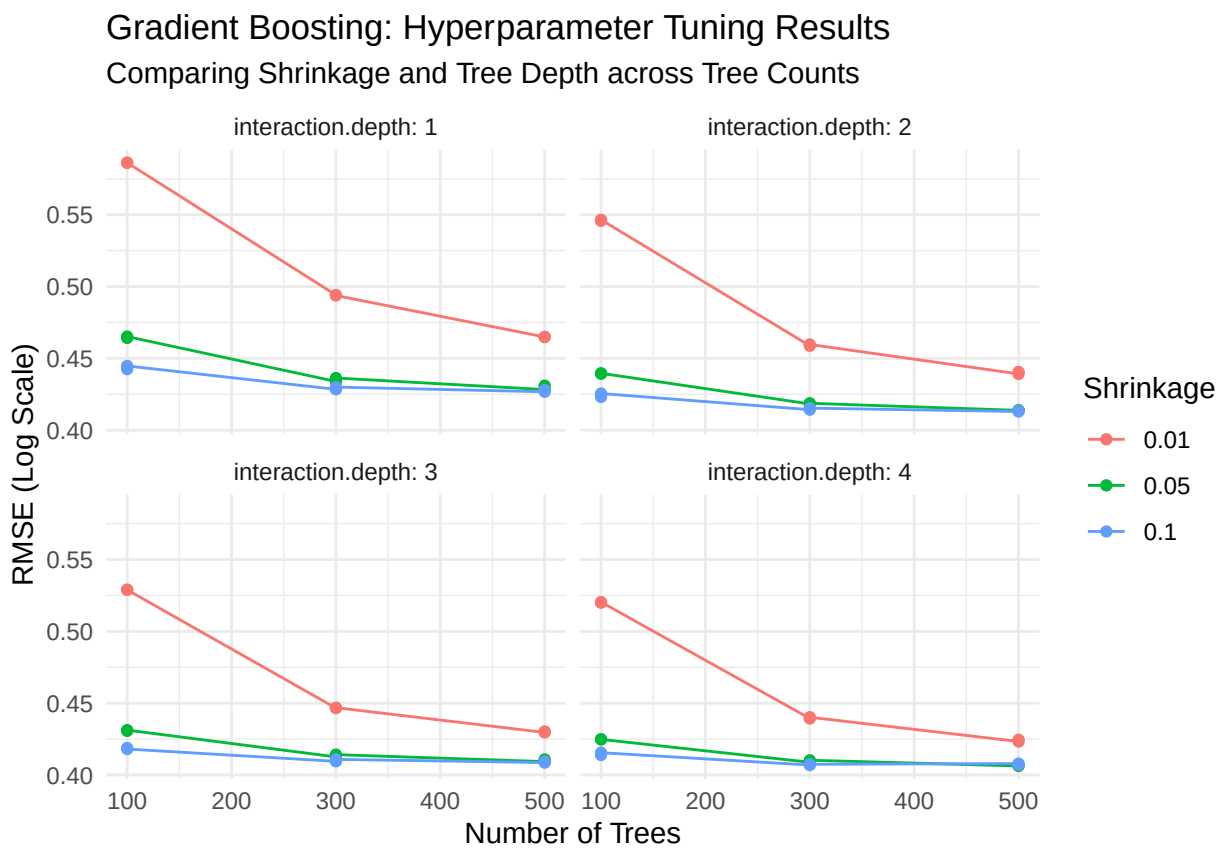
```
##    shrinkage interaction.depth n.minobsinnode n.trees      RMSE  Rsquared
## 1       0.05                 4              5     500 0.4062794 0.7029403
## 2       0.05                 4             10     500 0.4068951 0.7021799
## 3       0.10                 4              5     300 0.4069369 0.7016946
## 4       0.10                 4             10     500 0.4071200 0.7021838
## 5       0.10                 4             10     300 0.4074532 0.7011538
## 6       0.10                 4              5     500 0.4079541 0.7007366
## 7       0.10                 3              5     500 0.4086440 0.6995494
## 8       0.05                 4              5     300 0.4088653 0.6989549
## 9       0.05                 3              5     500 0.4093031 0.6982866
## 10      0.10                 3             10     500 0.4093868 0.6982581
##          MAE     RMSESD RsquaredSD      MAESD
## 1  0.3055876 0.03403964 0.03756759 0.02135392
## 2  0.3059675 0.03411732 0.03648934 0.02056838
## 3  0.3065322 0.03286144 0.03522134 0.01995735
## 4  0.3048217 0.03301641 0.03638297 0.01847796
## 5  0.3050816 0.03443985 0.03832812 0.02038391
## 6  0.3074074 0.03203798 0.03486406 0.01829378
## 7  0.3071364 0.03096248 0.03088355 0.01892239
## 8  0.3082549 0.03419026 0.03817250 0.02205120
## 9  0.3097136 0.03147880 0.03380051 0.01916008
## 10 0.3086176 0.03336659 0.03591623 0.01839390
```

```r
# Extract variable importance from the final GBM model
importance <- summary(gbm_cv$finalModel, plotit = FALSE)
# Filter for the top 10 most significant variables
top_performers <- head(importance, 10)
# Print the top 10 list
```

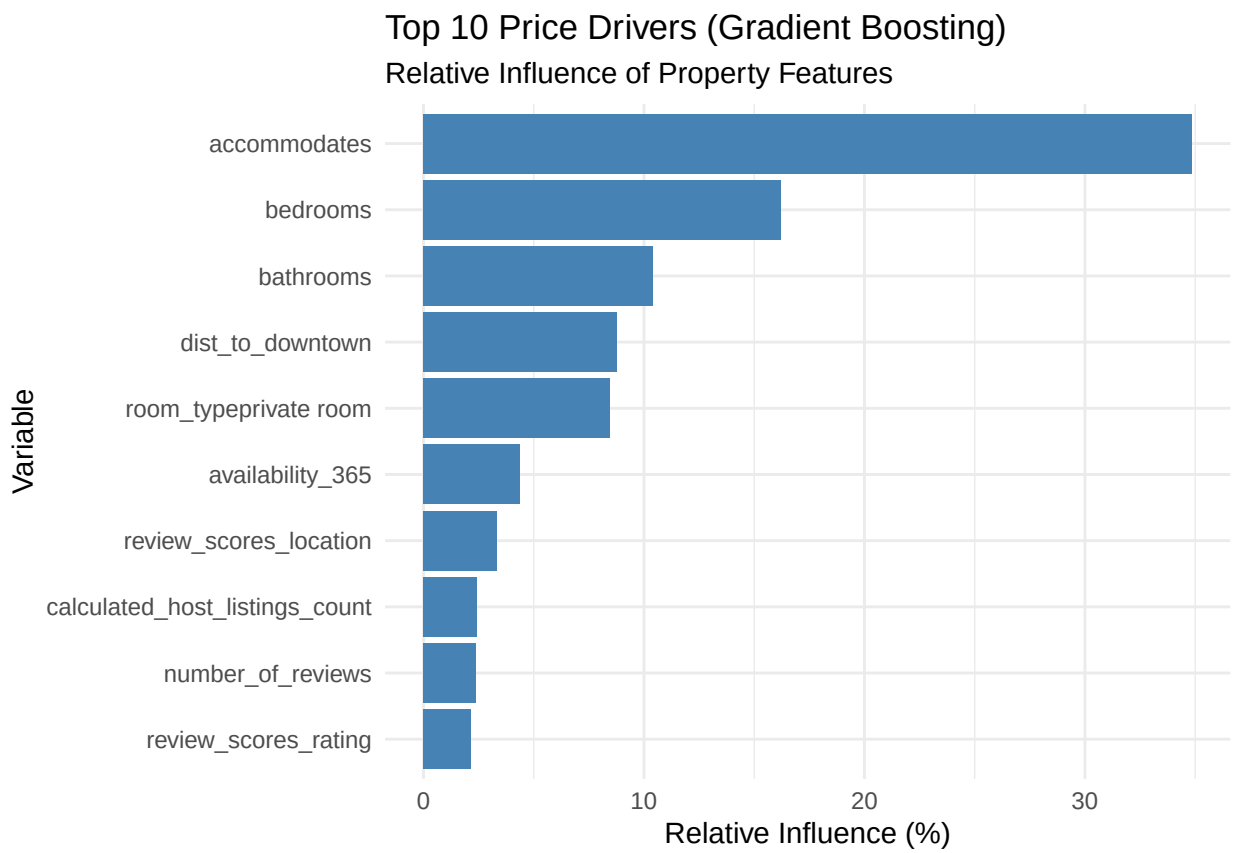
```r
print(top_performers)
```

```
##                                                          var   rel.inf
## accommodates                                   accommodates 34.852818
## bedrooms                                           bedrooms 16.197045
## bathrooms                                         bathrooms 10.412468
## dist_to_downtown                            dist_to_downtown  8.762016
## room_typeprivate room                   room_typeprivate room  8.452298
## availability_365                             availability_365  4.378063
## review_scores_location                 review_scores_location  3.353195
## calculated_host_listings_count calculated_host_listings_count  2.443310
## number_of_reviews                           number_of_reviews  2.367990
## review_scores_rating                     review_scores_rating  2.157779
```

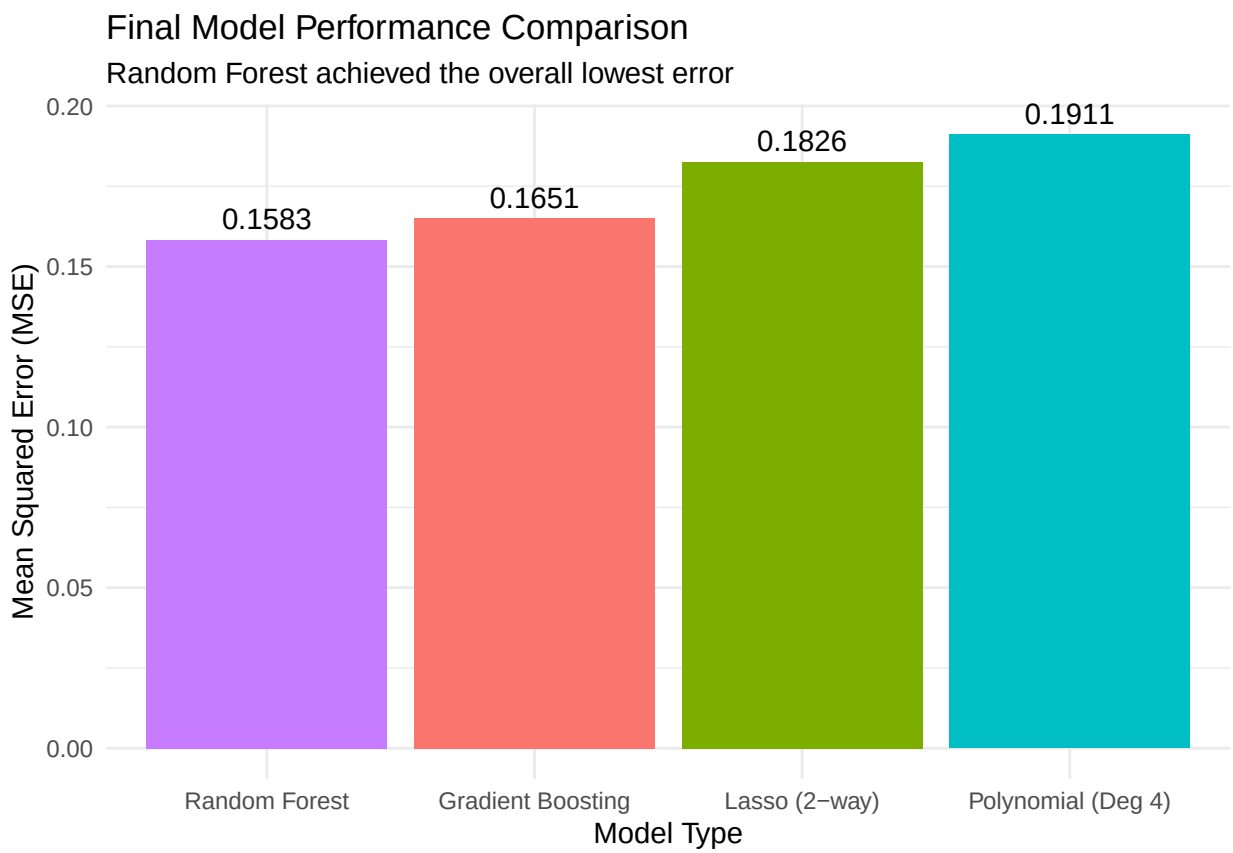
```r
#--------------Visualization-----------------
# Visualization of GBM Tuning results
ggplot(gbm_cv$results, aes(x = n.trees, y = RMSE, color = as.factor(shrinkage))) +
  geom_line() +
  geom_point() +
  facet_wrap(~interaction.depth, labeller = label_both) +
  labs(title = "Gradient Boosting: Hyperparameter Tuning Results",
       subtitle = "Comparing Shrinkage and Tree Depth across Tree Counts",
       x = "Number of Trees",
       y = "RMSE (Log Scale)",
       color = "Shrinkage") +
  theme_minimal()
```



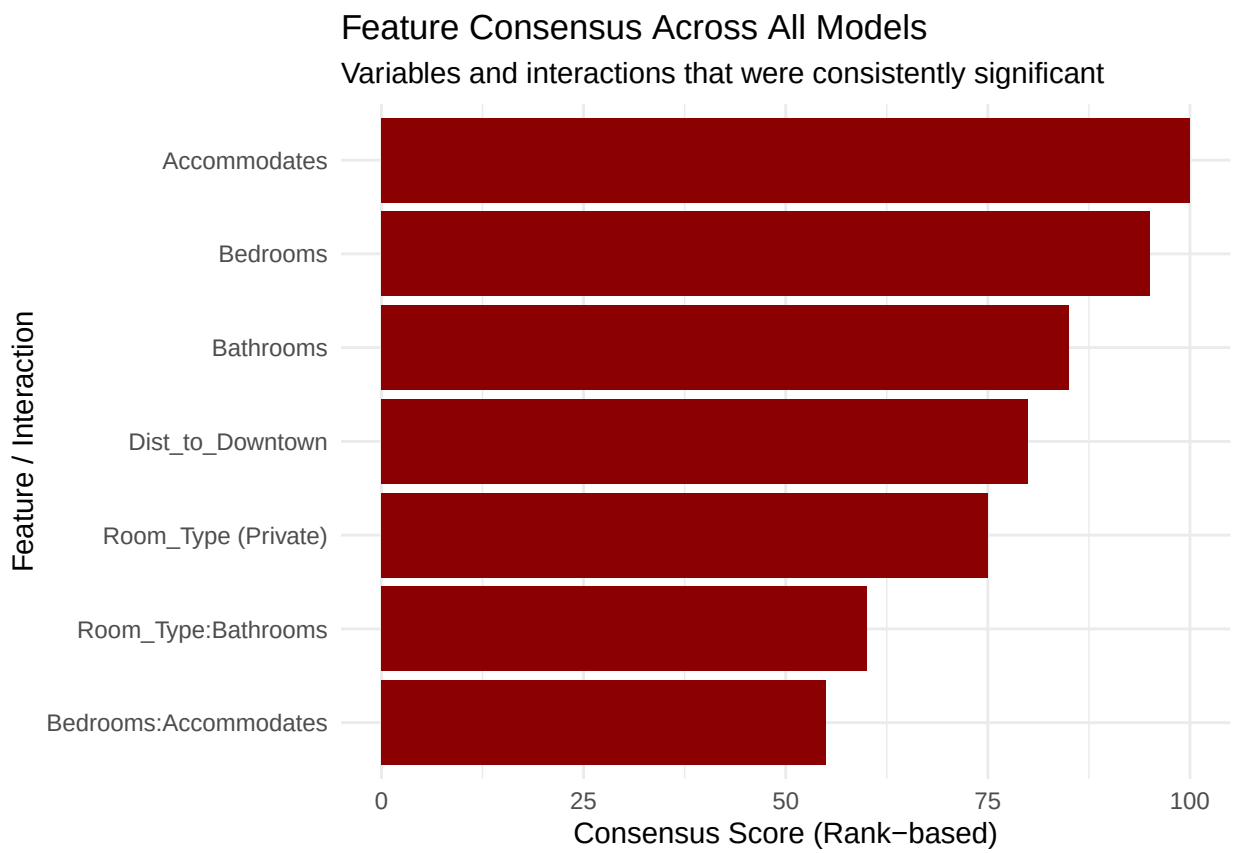

```r
#Non tech report
# Extract and Plot Variable Importance
# summary() for gbm creates a data frame of importance automatically
gbm_importance <- summary(gbm_cv$finalModel, plotit = FALSE)

ggplot(head(gbm_importance, 10), aes(x = reorder(var, rel.inf), y = rel.inf)) +
  geom_bar(stat = "identity", fill = "steelblue") +
  coord_flip() +
  labs(title = "Top 10 Price Drivers (Gradient Boosting)",
       subtitle = "Relative Influence of Property Features",
       x = "Variable",
       y = "Relative Influence (%)") +
  theme_minimal()
```




```r
library(ggplot2)
#The big picture
model_comparison <- data.frame(
  Model = c("Polynomial (Deg 4)", "Lasso (2-way)", "Random Forest", "Gradient Boosting"),
  MSE = c(0.1911, 0.1826, 0.1583, 0.1651)
)

ggplot(model_comparison, aes(x = reorder(Model, MSE), y = MSE, fill = Model)) +
  geom_bar(stat = "identity") +
  geom_text(aes(label = round(MSE, 4)), vjust = -0.5) +
  labs(title = "Final Model Performance Comparison",
       subtitle = "Random Forest achieved the overall lowest error",
       x = "Model Type",
       y = "Mean Squared Error (MSE)") +
  theme_minimal() +
  guides(fill = "none")
```




```r
#--------Important model features--------------
# Create a consensus data frame based on your combined outputs
consensus_data <- data.frame(
  Feature = c("Accommodates", "Bedrooms", "Bathrooms", "Dist_to_Downtown",
              "Room_Type (Private)", "Room_Type:Bathrooms", "Bedrooms:Accommodates"),
  Agreement_Score = c(100, 95, 85, 80, 75, 60, 55) # Simplified representation of rank overlap
)

ggplot(consensus_data, aes(x = reorder(Feature, Agreement_Score), y = Agreement_Score)) +
  geom_bar(stat = "identity", fill = "darkred") +
  coord_flip() +
  labs(title = "Feature Consensus Across All Models",
       subtitle = "Variables and interactions that were consistently significant",
       x = "Feature / Interaction",
       y = "Consensus Score (Rank-based)") +
  theme_minimal()
```


Feature Consensus Across All Models
Variables and interactions that were consistently significant
Feature / Interaction
Accommodates
Bedrooms
Bathrooms
Dist_to_Downtown
Room_Type (Private)
Room_Type:Bathrooms
Bedrooms:Accommodates
0
25
50
75
100
Consensus Score (Rank−based)